First Two-Class Appointment Simulation: Recommended Formulation

Alexandre SEPULVEDA de DIETRICH
Under the supervision of Pr. Carri Chan

April 9, 2026

Contents

1 Purpose and Model at a Glance **2**
1.1 Notation Choice . . . 2

2 Core Formulation **3**
2.1 Time Index . . . 3
2.2 State Representation . . . 4
2.3 Patient Classes and Arrival Mix . . . 5

3 Dynamics and Baseline Policy **6**
3.1 Slot-Level Event Order . . . 6
3.2 FCFS Baseline Policy . . . 7

4 Worked Example **7**

5 Performance Measures **9**
5.1 Arrival-Cohort Measures . . . 9

6 Relation to the Literature **10**
6.1 Literature Mapping . . . 10

7 Implementation Notes and Current Limits **12**
7.1 Implementation Mapping . . . 12
7.2 Current Scope and Limits . . . 12

8 References **13**

1 Purpose and Model at a Glance

This note defines the first discrete-event appointment simulation used in this project and clarifies the notation so it matches the underlying dynamics of the problem. The main design goal is to keep the first simulator simple enough to run, inspect, and extend, while using a formulation that remains valid once we add richer allocation policies, heterogeneous patient behavior, or optimization layers.

The note aligns the simulator with the queueing models in:

- Green and Savin (2008), *Reducing Delays for Medical Appointments: A Queueing Approach*,
- Liu and Ziya (2014), *Panel Size and Overbooking Decisions for Appointment-based Services under Patient No-shows*,
- Liu (2015), *Optimal Choice for Appointment Scheduling Window under Patient No-Show Behavior*.

All three papers study the same structural mechanism:

Core Feedback Loop

more backlog $\Rightarrow$ more delay $\Rightarrow$ worse attendance $\Rightarrow$ lower effective capacity.

The simulation keeps that mechanism, but implements it on an explicit rolling calendar with two patient classes, explicit cancellations, and a pluggable slot-allocation rule.

Model at a Glance

- The simulator evolves at slot-sized decision epochs $t = (D, s)$, where D is the day and s is the active slot within that day.
- The true state is the rolling calendar $Y_t(r, m)$, which records the content of slot m on day offset r.
- The derived day-start summary $X^D_{i,r}$ counts how many class-i patients are scheduled for day $D + r$ at the start of day D.
- Patient behavior is driven by the delay τ seen when the slot is offered, through $b_i(\tau)$, $\phi_i(\tau)$, and $\xi_i(\tau)$.
- Demand is parameterized by total arrival rate λ and class-1 demand share p, with $\lambda_1 = p\lambda$ and $\lambda_2 = (1 - p)\lambda$.

The note proceeds in six steps. It first states the modeling choice and notation, then defines the core formulation, then describes the baseline dynamics, then walks through one integrated FCFS example, then defines the performance measures, and finally positions the simulator relative to the literature and the current codebase.

1.1 Notation Choice

A naive notation might write

$$X^D_{i,\tau} = \text{number of class-}i\text{ patients booked for day } D + \tau$$

and then also use the same symbol τ inside $b_i(\tau)$ and $\xi_i(\tau)$. This note avoids that formulation for three reasons:

1. it overloads the delay variable, because the simulator needs both a *residual delay* in the state and a *booking-time delay* in the behavior functions;
2. it treats the aggregate count $X_{i,r}^{D}$ as if it were the full state, even though the simulator must track slot-level holes and positions explicitly;
3. it obscures the link to the literature, where attendance depends on the congestion or delay seen when the appointment is accepted.

The resulting modeling choice is simple: use r for residual delay in the state, use τ for the booking-time delay faced by the patient, and treat the slot-level calendar $Y_t(r, m)$ as the true state of the simulator.

2 Core Formulation

2.1 Time Index

Let time advance in slot-sized decision epochs. Write

$$t = (D, s),$$

where:

- D is the calendar day,
- $s \in \{0, 1, \ldots, S-1\}$ is the active slot within day D.

In the first version of the simulator,

$$H = 15 \text{ days}, \qquad S = 25 \text{ slots per day},$$

where H is the booking horizon.

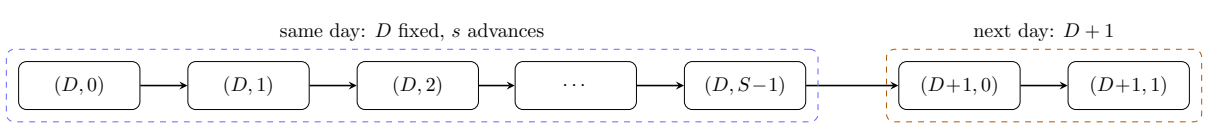
2.1.1 How Time Advances

The ordered pair $t = (D, s)$ advances lexicographically:

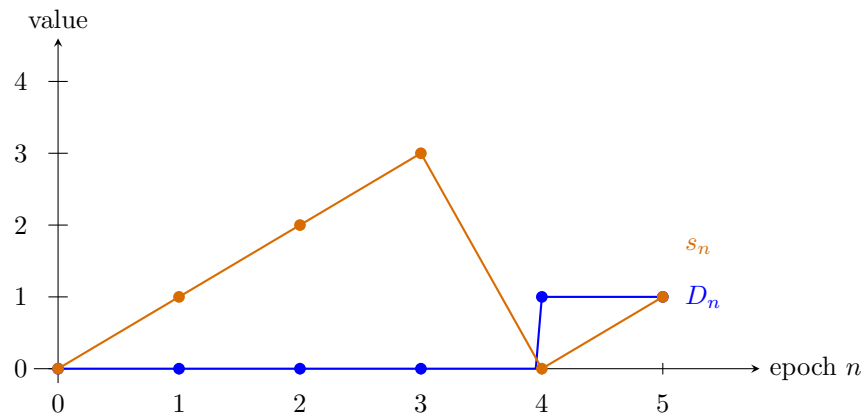
$$(D, s)^{+} = \begin{cases} (D, s+1), & \text{if } s < S-1, \\ (D+1, 0), & \text{if } s = S-1. \end{cases}$$

So the slot index s moves within a day, and the day index D increases only when the current day has exhausted all S slots.

The same evolution can be visualized as a state-transition graph:



To see both coordinates on the same graph, it is useful to plot them against the epoch index. The following illustration starts at day 0 and uses $S = 4$ only so the rollover is visible on a small plot:



On this graph, D_n is a step process while s_n increases within a day and resets to 0 when the process rolls to the next day.

This is the time scale on which arrivals, booking decisions, cancellations applied at the start of each day, and service realizations are processed.

2.2 State Representation

The true simulator state at time $t = (D, s)$ is the rolling slot-level calendar. The mathematical state is indexed by residual day r and slot index m. For visualization, however, it is often more natural to display the calendar with days as *columns* and slots as *rows*. In that display, the column index corresponds to the day ahead relative to the current day D: $r = 0$ means "today", $r = 1$ means "tomorrow", and so on. The row index corresponds to the slot within a day: $m = 0$ is the first slot, $m = 1$ the second, and so on until $m = S - 1$.

Formally, for each residual day offset

$$r \in \{0, 1, \ldots, H - 1\}$$

and each slot index

$$m \in \{0, 1, \ldots, S - 1\},$$

define

$$Y_t(r, m).$$

So $Y_t(r, m)$ is the content of the cell indexed by day-ahead r and slot m in the rolling calendar.

$$Y_t(r, m) = \begin{cases} 0, & \text{if the slot is open,} \\ (i, \tau)\,, & \text{if the slot is occupied by a class-}i\text{ patient who originally accepted delay } \tau. \end{cases}$$

The class index i links that booked patient to the class object, written later as

$$C_i = \{\lambda_i, b_i(\tau), \phi_i(\tau), \xi_i(\tau)\}.$$

So the cell itself stores only the patient features that must remain attached to the appointment: the class identity i and the original offered delay τ. The remaining class characteristics are recovered from C_i when the simulator needs them:

- λ_i governs future arrivals from class i,
- $b_i(\tau)$ governs booking acceptance when a class-i request is offered delay τ,
- $\phi_i(\tau)$ governs daily pre-appointment cancellation for class-i patients booked with delay τ,
- $\xi_i(\tau)$ governs no-show at appointment time.

2.2.1 Derived Day-Start Summary

Although the slot-level calendar is the true state, it is still useful to retain an aggregate day-start summary that resembles the literature. At the start of day D, after the previous day's end-of-day cancellations and before processing slot 0, define

$$X_{i,r}^D = \sum_{m=0}^{S-1} \mathbf{1}\{Y_{(D,0)}(r,m) \text{ is occupied by a class-}i \text{ patient}\},$$

for

$$i \in \{1,2\}, \qquad r \in \{0,1,\ldots,H-1\}.$$

In words, $X_{i,r}^D$ counts how many class-i patients appear in row r of the calendar at the start of day D. Equivalently,

$$X_{i,r}^D = \text{number of class-}i \text{ patients scheduled at the start of day } D \text{ for day } D+r.$$

This is the correct interpretation of the state-count variable. The index r is a *residual delay*, not a booking-time delay.

2.2.2 Booking-Time Delay

Now define the behavioral delay variable separately:

$$\tau = \text{delay in days between the booking day and the appointment day offered to the patient.}$$

If a patient arrives on day d and is offered a slot on day $d+\tau$, then:

- $b_i(\tau)$ is the probability that class-i declines the offer,
- if the patient accepts, τ is stored with the booking,
- $\xi_i(\tau)$ is the no-show probability used at appointment time if the patient remains scheduled.

Key Modeling Distinction

Residual delay r belongs to the state. Booking-time delay τ belongs to the behavioral rules.

2.3 Patient Classes and Arrival Mix

The class-specific behavior of each class $i \in \{1,2\}$ is described by

$$B_i = \{b_i(\tau), \phi_i(\tau), \xi_i(\tau)\}.$$

In the current two-class prototype, arrivals are parameterized at the system level by

$$\lambda = \text{total expected arrival rate per slot}, \qquad p \in [0,1] = \text{share of class-1 demand.}$$

The class-specific arrival rates are then induced by

$$\lambda_1 = p\lambda, \qquad \lambda_2 = (1-p)\lambda.$$

Equivalently, the full class objects can still be written as

$$C_i = \{\lambda_i, b_i(\tau), \phi_i(\tau), \xi_i(\tau)\},$$

but in the first simulation experiments the primitive demand parameters are (λ, p) rather than (λ_1, λ_2).

The interpretation is therefore:

- λ: total expected arrival rate per slot,
- p: fraction of arrivals belonging to class 1,
- λ_i: implied arrival rate of class-i requests per slot,
- $b_i(\tau)$: probability the patient declines the offered slot because the offered delay is too large,
- $\phi_i(\tau)$: daily probability that a class-i patient booked with original delay τ cancels at the end of a pre-appointment day,
- $\xi_i(\tau)$: no-show probability at appointment time, conditional on the patient still being scheduled.

In the current first version, the daily cancellation function $\phi_i(\tau)$ is induced by a class-level eventual cancellation parameter $\bar{\phi}_i$ according to

$$\phi_i(\tau) = \begin{cases} 0, & \tau = 0, \\ 1 - (1-\bar{\phi}_i)^{1/\tau}, & \tau \geq 1, \end{cases}$$

so that a patient who booked with delay τ and faces one cancellation trial at the end of each pre-appointment day still has total cancellation probability $\bar{\phi}_i$ before the appointment. Later versions may generalize this further to a directly delay-sensitive cancellation object.

3 Dynamics and Baseline Policy

3.1 Slot-Level Event Order

At each active slot (D, s), the simulator executes the following within-day steps.

1. Generate arrivals
$$A_i(D, s) \sim \text{Poisson}(\lambda_i), \qquad i \in \{1, 2\}.$$
In the current parameterization, $\lambda_1 = p\lambda$ and $\lambda_2 = (1-p)\lambda$.
2. Randomize the order of individual arrivals within the slot.
3. For each arrival, call the allocation policy to select an open future slot.
4. If a slot on day $D + r$ is offered, interpret
$$\tau = r.$$
5. The patient accepts with probability
$$1 - b_i(\tau).$$

6. When the active slot is served, the patient in that slot, if any, attends with probability

$$1 - \xi_i(\tau),$$

where τ is the original delay stored when the patient booked.

After all S slots of day D have been processed, every still-scheduled future appointment stored as (i, τ) cancels independently with probability $\phi_i(\tau)$. Any canceled future slot is therefore reopened before day $D + 1$ begins. This timing is operationally natural:

- a booking with $\tau = 1$ gets exactly one chance to cancel, namely at the end of the booking day,
- a booking with $\tau = 2$ gets two daily cancellation opportunities before service,
- the total cancellation probability across those daily trials remains $\bar{\phi}_i$.

3.2 FCFS Baseline Policy

In the baseline version, the allocation rule is:

choose the earliest open slot strictly after the active slot.

That rule is first-come first-served in calendar order. In this note it is the only policy under study, because the goal is to isolate the basic delay-feedback mechanism before considering richer allocation logic.

4 Worked Example

The abstract notation is easier to read once it is tied to a single concrete FCFS calendar. This section combines the visual calendar, the detailed state $Y_t(r, m)$, the derived count $X^D_{i,r}$, and the implied booking decision in one place.

Visualizing the Rolling $H \times S$ Calendar

Before filling in a specific scenario, it helps to see the shape of the state. The rolling calendar has the following generic structure:

Slot in day	**day** D $(r = 0)$	**day** $D + 1$ $(r = 1)$	$\cdots$	**day** $D + H - 1$ $(r = H - 1)$
slot 0 $(m = 0)$	$Y_t(0, 0)$	$Y_t(1, 0)$	$\cdots$	$Y_t(H - 1, 0)$
slot 1 $(m = 1)$	$Y_t(0, 1)$	$Y_t(1, 1)$	$\cdots$	$Y_t(H - 1, 1)$
later slots	$\vdots$	$\vdots$	$\ddots$	$\vdots$
slot $S - 1$ $(m = S - 1)$	$Y_t(0, S - 1)$	$Y_t(1, S - 1)$	$\cdots$	$Y_t(H - 1, S - 1)$

Each column is a day in the rolling horizon, and each row is a slot inside that day. The state variable $Y_t(r, m)$ is simply the content of one cell of this table.

Worked Example: FCFS Calendar, Derived State, and Booking Decision

Suppose class 1 is an MRI-like diagnostic request and class 2 is a behavioral-health follow-up request. Suppose also that each day has four appointment times: 08:00, 10:00, 13:00, and 15:00. Take

$$D = 0, \qquad s = 0, \qquad H = 3, \qquad S = 4,$$

so we are at the start of the current operating day, just before the 08:00 slot, and the rolling calendar covers today, tomorrow, and the day after tomorrow.
A realistic FCFS-consistent miniature appointment book looks like this:

Slot in day	**day D**	**day $D+1$**	**day $D+2$**
08:00 $(m = 0)$	$(1, 0)$	$(2, 1)$	$(2, 2)$
10:00 $(m = 1)$	$(2, 0)$	$(1, 1)$	open
13:00 $(m = 2)$	$(1, 0)$	$(2, 1)$	open
15:00 $(m = 3)$	open	$(1, 1)$	open

This example is read column by column. Today and tomorrow are filled first, overflow appears on day $D + 2$, and the open 15:00 slot on day D is a cancellation hole rather than a violation of FCFS.
The same calendar can be written directly as a state snapshot:

	08:00	10:00	13:00	15:00
$r = 0$ (today)	$(1, 0)$	$(2, 0)$	$(1, 0)$	0
$r = 1$ (tomorrow)	$(2, 1)$	$(1, 1)$	$(2, 1)$	$(1, 1)$
$r = 2$ (day after)	$(2, 2)$	0	0	0

So, for example:

- $Y_{(0,0)}(0, 0) = (1, 0)$ means that today's 08:00 slot is booked by a class-1 MRI patient who accepted a same-day offer;
- $Y_{(0,0)}(1, 3) = (1, 1)$ means that tomorrow's 15:00 slot is booked by a class-1 MRI patient who accepted a one-day delay;
- $Y_{(0,0)}(2, 0) = (2, 2)$ means that the 08:00 slot two days from now is booked by a class-2 behavioral-health patient who accepted a two-day delay.

Because the last patient belongs to class 2, the simulator links that slot to

$$C_2 = \{\lambda_2, b_2(\tau), \phi_2(\tau), \xi_2(\tau)\},$$

so future class-2 arrivals are generated with rate λ_2, daily cancellation while the patient waits is governed by $\phi_2(2)$, and eventual no-show is governed by $\xi_2(2)$.
From the same calendar, the derived day-start counts are:

$$X^0_{1,0} = 2, \quad X^0_{2,0} = 1, \qquad X^0_{1,1} = 2, \quad X^0_{2,1} = 2, \qquad X^0_{1,2} = 0, \quad X^0_{2,2} = 1.$$

So $X^0_{i,r}$ is just a class-by-class count of occupied entries in row r. It does *not* remember which slot inside the day is occupied, which is why $X^D_{i,r}$ is a useful summary but not the full simulator state.
The same snapshot also shows why r and τ must be separated. The patient stored as

$$Y_{(0,0)}(2,0) = (2,2)$$

contributes to $X^0_{2,2}$ on day 0, then to $X^1_{2,1}$ on day 1 if still scheduled. The residual delay changes from $r = 2$ to $r = 1$, but the booking-time delay remains $\tau = 2$, so the same behavioral functions $b_2(\tau)$, $\phi_2(\tau)$, and $\xi_2(\tau)$ continue to apply.
Finally, the calendar transfers directly into the booking equations. Suppose a new patient arrives just before today's 08:00 slot. Under FCFS, the earliest open slot is today's 15:00 slot. If the arrival belongs to class 1, then:

- the offered delay is $\tau = 0$;
- the patient books with probability $1 - b_1(0)$;
- if the patient accepts, the new slot content becomes $Y_{(0,0)}(0,3) = (1,0)$;
- because $\tau = 0$, there is no remaining pre-appointment day on which cancellation can occur;
- if the patient reaches the appointment, attendance occurs with probability $1 - \xi_1(0)$.

So the same worked example can be read in three ways: as an appointment book, as a state realization (Y, X), and as the input to the booking and attendance equations.

5 Performance Measures

5.1 Arrival-Cohort Measures

For patients who arrive during the measured window, define:

Measure	Definition
Arrivals_i	number of class-i appointment requests
Booked_i	number of class-i requests that accept an offer
Balked_i	number of class-i requests that were offered a slot and declined it
NoOffer_i	number of class-i requests for which no eligible slot was available
NotBooked_i	$\text{Balked}_i + \text{NoOffer}_i$
Canceled_i	number of class-i booked patients who pre-cancel
NoShow_i	number of class-i booked patients who remain scheduled but do not attend
Served_i	number of class-i booked patients who attend
$\bar{\tau}_i^{\text{book}}$	mean original booking delay among class-i booked patients

The main service metric in the first version is

$$\text{ServiceRate}_i = \frac{\text{Served}_i}{\text{Booked}_i}.$$

Aggregate values are formed by summing over classes and taking the corresponding weighted mean.

5.1.1 Slot-Based Measures

Over measured service slots, define:

$$\begin{aligned}
\text{BookedSlots} &= \text{slots with a booked patient at service time,}\\
\text{ServedSlots} &= \text{slots where the booked patient attends,}\\
\text{NoShowSlots} &= \text{slots where a booked patient is present on the schedule but does not attend,}\\
\text{EmptySlots} &= \text{slots with nobody scheduled at service time.}
\end{aligned}$$

Two utilization measures are reported:

$$\rho^{\text{booked}} = \frac{\text{BookedSlots}}{HS_{\text{measured}}}, \qquad \rho^{\text{attended}} = \frac{\text{ServedSlots}}{HS_{\text{measured}}}.$$

The first measures scheduled load. The second measures realized attended load.

Operationally, mean booking delay summarizes access, ρ^{booked} summarizes scheduled load, and ρ^{attended} summarizes realized clinical load after no-shows.

6 Relation to the Literature

6.1 Literature Mapping

The source papers use more stylized queue states than the simulator, but the economic and behavioral interpretation is the same. In each paper, attendance depends on the congestion level a patient sees when entering the system.

In the analytical papers, that congestion variable is usually:

- queue length,
- backlog level,
- or the number of appointments already ahead in the system.

In the simulator, the explicit-calendar analog is the offered delay τ. That is why τ, rather than residual delay r, should drive $b_i(\cdot)$ and $\xi_i(\cdot)$.

6.1.1 Green and Savin (2008)

Green and Savin study a single-server appointment queue in which no-show probability increases with the backlog seen when the appointment is made. Their no-show curve is of the exponential-saturation form

$$\gamma(k) = \gamma_{\max} - (\gamma_{\max} - \gamma_0)e^{-k/C},$$

where k is a backlog measure.

The mapping to the simulator is:

- their backlog state corresponds to our explicit slot calendar,
- their delay-sensitive no-show mechanism corresponds to our $\xi_i(\tau)$,
- their stylized queue-length state is refined into the explicit schedule $Y_t(r, m)$.

The simulation differs from Green and Savin in several ways:

- it allows two patient classes rather than one homogeneous stream,
- it includes explicit balking before booking,
- it includes daily pre-appointment cancellation that creates explicit holes in the calendar,
- it does not yet model no-show rescheduling.

6.1.2 Liu and Ziya (2014)

Liu and Ziya study panel size and overbooking decisions in stylized queueing models with delay-dependent attendance. Their state variable is again a queue-length-type object, and a patient's show-up probability depends on the congestion level observed at booking time.

Relative to Liu and Ziya, the simulator:

- fixes daily slot capacity at 25 in the first version,
- does not yet optimize overbooking or panel size,
- keeps the booking calendar explicit rather than compressing it into a single queue-length state,
- is designed to allow class-specific balking, cancellation, and attendance curves.

6.1.3 Liu (2015)

Liu studies the optimal appointment-window size K under delay-dependent no-shows. That paper's main control is the admissible backlog or booking window.

The simulator's rolling horizon length H plays the same operational role as a hard scheduling window, but here the horizon is fixed in the baseline implementation. This makes the simulator a natural platform for later experiments in which:

- H is varied,
- different allocation policies are compared,

- or class-specific admission rules are introduced.

6.1.4 Side-by-Side Comparison

Model / paper	State abstraction	Delay variable that drives attendance	Main difference from this simulator
Green and Savin (2008)	queue length / backlog in an appointment queue	backlog seen when appointment is made	analytical single-class queue; no explicit slot calendar
Liu and Ziya (2014)	queue length in $M/M/1$ or $M/D/1$ style models	congestion at booking time	optimization over panel size and overbooking, not explicit calendar simulation
Liu (2015)	queue/window state with finite scheduling window K	delay implied by admissible backlog at booking	optimization over window size rather than fixed-horizon simulation
This simulator	explicit rolling calendar $Y_t(r, m)$ plus derived $X^D_{i,r}$	original offered delay τ	richer operational detail, two classes, daily cancellation holes, policy plug points

7 Implementation Notes and Current Limits

7.1 Implementation Mapping

The current codebase follows this notation directly:

- the slot calendar stores each patient's class, appointment location, booking day, and the mathematical delay τ,
- the arrival log records the offered delay as `tau_offered`,
- the cohort and slot logs record the same mathematical quantity τ under the implementation field name `tau_booked`,
- the day-start state log records the residual day offset as `residual_delay`,
- the class configuration field `cancel_probability` stores the class-level eventual cancellation parameter $\bar{\phi}_i$, from which the daily function $\phi_i(\tau)$ is derived in code.

This is the same split as in the mathematical formulation. In addition, the simulator now exposes a daily journal with access, booking, cancellation, attendance, and utilization measures, and reports the access metric

$$\mathbb{P}(\tau < T_{\text{access}})$$

for a configurable target such as $T_{\text{access}} = 30$ days.

7.2 Current Scope and Limits

FCFS is the deliberate first baseline because it makes the delay-feedback mechanism easy to see:

$$\text{more demand} \longrightarrow \text{longer offered delay } \tau \longrightarrow \text{more balking / cancellation / no-show.}$$

That makes it a good diagnostic starting point, even though it should not be interpreted as the final operational recommendation when classes differ in delay sensitivity.

The implementation is also intentionally a little richer than the minimal mathematical state. The calendar keeps additional audit information so logs and journals can be produced, but the key behavioral content still remains the pair (i, τ). The main provisional simplification is cancellation: mathematically the note works with $\phi_i(\tau)$, while in code the current class configuration still starts from a scalar parameter $\bar{\phi}_i$ and derives the daily cancellation function internally. That is acceptable for the first prototype, but the natural next step is to let the code accept a cancellation function directly if future data support that level of estimation.

8 References

- Green, L. V., and S. Savin. 2008. Reducing delays for medical appointments: A queueing approach. *Operations Research* 56(6):1526–1538.
- Liu, N. 2015. Optimal choice for appointment scheduling window under patient no-show behavior. *Production and Operations Management* 25(1):128–142.
- Liu, N., and S. Ziya. 2014. Panel size and overbooking decisions for appointment-based services under patient no-shows. *Production and Operations Management* 23(12):2209–2223.